Design using System Verilog

Assignment_5(Assignment_5 written in classroom)

- Toshiv Kumar, 1024060266, 3F32, ECE

-

Aim:

- Write a SystemVerilog function to add, subtract, multiply, and divide two integers.
- Invoke the function from an initial block and display the results.
- Understand function declaration, arguments, return type, and function invocation.
- Create a function that accepts three inputs and returns the .
- Modify the function to determine whether an input number is .
- Use functions to model reusable combinational operations.
- Write a void function that accepts student marks in three subjects and displays the .
- Since a void function does not return a value, display the result from within the function.
- Understand the difference between a value-returning function and a void function.
- Write a SystemVerilog task that accepts two operands and performs .
- Return/display all four results from the task.
- Understand task declaration, input/output arguments, and multiple outputs.
- Develop a task to generate a sequence of test inputs for a .
- Include appropriate # delays between successive input combinations.
- Understand why tasks are useful for testbench stimulus generation and how they differ from functions.
- Implement the same arithmetic operation using both a .
- Compare them based on:
- Return value
- Number/type of arguments
- Timing control
- Typical application

Identify when to use a task and when to use a function.

- Design a task to apply different combinations of inputs to a and automatically display:
- Inputs A, B, Cin
- Expected Sum, Cout
- Actual Sum, Cout

Use tasks to create reusable and organized verification code.

- Create a function that calculates the .
- Use the function in a testbench to compare the expected output with the DUT output and report .
- Understand the use of functions in automated verification.

Perform the following activities:

1. Reusable systemverilog functions for arithmetic, checking even/odd, and a void displa y function.

Code:

Design file:

TB file:

```
`timescale 1ns / 1ps


// Toshiv Kumar(NAME) , 1024060266(Roll Number)
module functions_demo;


    function automatic int add_ints(input int a, input int b);
        add_ints = a + b;
    endfunction


    function automatic int sub_ints(input int a, input int b);
```

```
    sub_ints = a - b;
endfunction

function automatic int mul_ints(input int a, input int b);
    mul_ints = a * b;
endfunction

function automatic int div_ints(input int a, input int b);
    div_ints = a / b;
endfunction

// a function returns one value.
// it is used for combinational work with no timing delay.

function automatic int sum3(input int a, input int b, input int c);
    sum3 = a + b + c;
endfunction

function automatic bit is_even(input int num);
    is_even = (num % 2 == 0);
endfunction

// void function does not return a value.
// the result is displayed from inside the function itself.

function void student_result(input int m1, input int m2, input int m3);
    int total;
    real avg;
    total = m1 + m2 + m3;
```

```
    avg = total / 3.0;
    $display("marks = %0d %0d %0d", m1, m2, m3);
    $display("total = %0d", total);
    $display("average = %0f", avg);
endfunction

initial begin
    int a, b, c;

    a = 20;
    b = 5;
    c = 3;

    $display("add = %0d", add_ints(a, b));
    $display("sub = %0d", sub_ints(a, b));
    $display("mul = %0d", mul_ints(a, b));
    $display("div = %0d", div_ints(a, b));
    $display("sum3 = %0d", sum3(a, b, c));

    if (is_even(10))
        $display("10 is even");
    else
        $display("10 is odd");

    student_result(78, 85, 91);

    $finish;
end
```

endmodule

Output/Screenshot:

```
# run 1000ns
add = 25
sub = 15
mul = 100
div = 4
sum3 = 28
10 is even
marks = 78 85 91
total = 254
average = 84.666667
```

2. Systemverilog tasks with multiple outputs and delayed stimulus generation.

Code:

Design file:

TB:

```
`timescale 1ns / 1ps


// Toshiv Kumar(NAME) , 1024060266(Roll Number)



module tasks_demo;


    task automatic four_ops(
        input  int a,
        input  int b,
        output int add_r,
        output int sub_r,
        output int mul_r,
        output int div_r
    );
        add_r = a + b;
        sub_r = a - b;
        mul_r = a * b;
```

```
        if (b != 0)
            div_r = a / b;
        else
            div_r = 0;
    endtask

    task automatic apply_inputs;
        int a, b, add_r, sub_r, mul_r, div_r;

        a = 10; b = 2;
        four_ops(a, b, add_r, sub_r, mul_r, div_r);
        $display("a=%0d b=%0d add=%0d sub=%0d mul=%0d div=%0d", a, b, add_r, sub_r, mul_r, div_r);
        #5;

        a = 15; b = 3;
        four_ops(a, b, add_r, sub_r, mul_r, div_r);
        $display("a=%0d b=%0d add=%0d sub=%0d mul=%0d div=%0d", a, b, add_r, sub_r, mul_r, div_r);
        #5;

        a = 8; b = 4;
        four_ops(a, b, add_r, sub_r, mul_r, div_r);
        $display("a=%0d b=%0d add=%0d sub=%0d mul=%0d div=%0d", a, b, add_r, sub_r, mul_r, div_r);
        #5;
    endtask

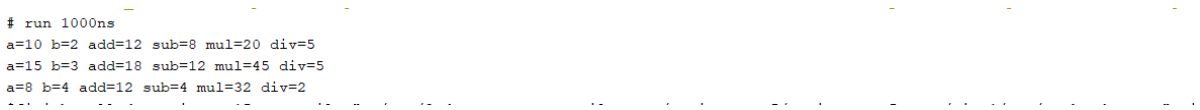
    // task vs function:
    // function:
```

```
    // - returns one value
    // - no timing control like #, @, wait
    // - used for combinational calculations
    //
    // task:
    // - can return multiple values using output/inout
    // - can use timing control
    // - used for stimulus generation and verification

    initial begin
        apply_inputs();
        $finish;
    end

endmodule
```

Output/Screenshot:



3. verified a full adder using a function for expected values and a task for input application and comparison.

Code:

Design file:

TB:

`timescale 1ns / 1ps

```
// Toshiv Kumar(NAME) , 1024060266(Roll Number)

module full_adder(
    input  logic a,
    input  logic b,
    input  logic cin,
    output logic sum,
    output logic cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (cin & a);
endmodule

module full_adder_check;

    logic a, b, cin;
    logic sum, cout;
    logic exp_sum, exp_cout;

    full_adder dut(
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .cout(cout)
    );

    function automatic void expected_full_adder(
        input logic a_f,
```

```
    input logic b_f,
    input logic cin_f,
    output logic sum_f,
    output logic cout_f
);
    sum_f = a_f ^ b_f ^ cin_f;
    cout_f = (a_f & b_f) | (b_f & cin_f) | (cin_f & a_f);
endfunction

task automatic apply_and_check(input logic a_t, input logic b_t, input logic cin_t);
    expected_full_adder(a_t, b_t, cin_t, exp_sum, exp_cout);

    a = a_t;
    b = b_t;
    cin = cin_t;

    #1;

    $display("inputs: a=%b b=%b cin=%b | expected: sum=%b cout=%b | actual: sum=%b
cout=%b",
            a, b, cin, exp_sum, exp_cout, sum, cout);

    if ((sum === exp_sum) && (cout === exp_cout))
       $display("pass");
    else
       $display("fail");

    #4;
endtask
```

```
    initial begin
        apply_and_check(0, 0, 0);
        apply_and_check(0, 0, 1);
        apply_and_check(0, 1, 0);
        apply_and_check(0, 1, 1);
        apply_and_check(1, 0, 0);
        apply_and_check(1, 0, 1);
        apply_and_check(1, 1, 0);
        apply_and_check(1, 1, 1);

        $finish;
    end

endmodule
```

Output/Screenshot:

```
inputs: a=0 b=0 cin=0 | expected: sum=0 cout=0 | actual: sum=0 cout=0
pass
inputs: a=0 b=0 cin=1 | expected: sum=1 cout=0 | actual: sum=1 cout=0
pass
inputs: a=0 b=1 cin=0 | expected: sum=1 cout=0 | actual: sum=1 cout=0
pass
inputs: a=0 b=1 cin=1 | expected: sum=0 cout=1 | actual: sum=0 cout=1
pass
inputs: a=1 b=0 cin=0 | expected: sum=1 cout=0 | actual: sum=1 cout=0
pass
inputs: a=1 b=0 cin=1 | expected: sum=0 cout=1 | actual: sum=0 cout=1
pass
inputs: a=1 b=1 cin=0 | expected: sum=0 cout=1 | actual: sum=0 cout=1
pass
inputs: a=1 b=1 cin=1 | expected: sum=1 cout=1 | actual: sum=1 cout=1
pass
```

Learnings from this Experiment:

There are many things that I learned from this experiment.

I learned how to write reusable systemverilog functions for arithmetic operations and checking conditions. i learned the difference between a function and a task, especially that

functions return one value and tasks can return multiple values and include timing control. i learned how to use a void function to display results from inside the function itself. i also learned how to use tasks for testbench stimulus generation, delayed input application, and automatic checking of dut outputs against expected values.

Thank you for the assignment.